EGC-211P
C++ BASICS AND CLASSES

Brief History of C++

- Developed by *Bjarne Stroustrup* during 1979-1984
- The name **C++** came around 1983. It's a pun from C's increment operator (++). So **C++ literally means "C plus one"** — a geeky way of saying "a better C".
- Despite newer alternatives, **it refuses to die**. In fact, it keeps evolving (**C++11**, C++14, C++17, C++20, C++23…), *each version somehow adding more ways to break your brain*.
- Stroustrup wanted three things:
  - **Performance like C** -- Fast, lean.
  - **Abstraction** -- So developers didn't need to track memory with spreadsheets.
  - **General-purpose programming** -- For real-world software: operating systems, compilers, games, simulations, nuclear reactors, etc.



byaa·nuh **strawz**·truhp

Bjarne Stroustrup's Quotes

"C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off".

C++ Basics

Simple C++ Program

```
#include <iostream>
using namespace std ;
int main( )
{
        int num ;
        cin >> num ;
        cout << num << endl ; // endl is like "\n"
         return 0;
}
```

- cin, cout instead of scanf( ), printf( )
- No .h extension for standard header inclusion
- using namespace std – necessary for cin, cout to work

Addresses (Recap)

All declared variables implicitly point to memory addresses

- Global and static variables are created in the data segment of memory
- Local variables are created on stack
- Objects and Structures are allocated on the heap (if created using ***new***)

```
int ten = 10 ;
cout << "address of i = " << &ten ;
```

Will generate a hexadecimal output that shows the address of variable “i”

Pointers (Recap)

- Pointers contain addresses of variables in memory
- A pointer needs to be “dereferenced” (e.g. *p) to get the variable
- Unlike C, pointers and integers are NOT interchangeable in C++

```
#include <iostream>
using namespace std ;
int main( )
{
        int ten = 10 ;
        int* p_ten ;
        int** p_p_ten ;
        p_ten = &ten ;
        p_p_ten = &p_ten ;
        cout << ten << *p_ten << **p_p_ten ;
}
```

References

- Reference must always be initialized
- Reference is just an alias to the object it was initialized with
- A reference is automatically dereferenced when needed
- Changing a reference changes the variable / object it refers to

```
int var = 10 ; // declare variable i of type int and initialize to 10
int& ref = var ; // ref is declared as a reference and initialized to variable var

cout << var << ref ; // ref is automatically dereferenced on use
                                                    // producing 10 10 as the output
ref = 20 ;   // changing a reference changes the variable i itself
cout << var << ref ; // output will be 20 20

var = 30 ;      // changing the variable value
cout << var << ref ;  // j is automatically dereferenced so output is 30 30
```

Pointer (House) & Reference (Mom)





Remember this about References

- A reference is a **const** pointer to the variable / object it represents
  (there is only one Mom for a child)
- There is no “null reference” (no Mom without a child)
  - Pointer can be "null" --> house where nobody lives
- You can create multiple references (aliases) to an object
- You cannot create
  - A reference to a reference
  - A pointer to a reference
  - Array of references

References

- Added to C++ because of difficulties with pointers
  - Check for NULL pointer is tedious
  - Pointers and integers are treated as same in C
  - Pointers may be reassigned to other variables in code
- Call by “value” vs. call by “reference”
- In C++, a reference is a **const** pointer to the variable / object it represents

More Example: return by reference

```cpp
#include <iostream>
using namespace std;

class Bank {
    double balances[3] = {1000, 2500, 5000};

public:
    // Return reference to a customer's balance
    double& accountBalance(int accountNo) {
        return balances[accountNo];
    }

    void showAccounts() {
        for (int i = 0; i < 3; i++)
            cout << "Account " << i
                 << " : INR " << balances[i] << endl;
    }
};
```

```cpp
int main() {
    Bank bank;

    cout << "Before transaction:\n";
    bank.showAccounts();

    // Deposit money directly into Account 1
    bank.accountBalance( accountNo: 1) += 1000;

    // Withdraw money from Account 2
    bank.accountBalance( accountNo: 2) -= 1000;

    cout << "\nAfter transaction:\n";
    bank.showAccounts();
}
```

Using Pointers & References

- Dynamic memory allocation, using malloc() or new --> pointer
- Store objects in array --> pointer
- Rest all, use reference --> especially when to pass large objects

Break

const Modifier

- *const* keyword when applied to a variable declaration means that the value of the variable will not be modified subsequently.
- A reference is a *const* pointer

```
int main() {
    int const i = 3; // value of i cannot be modified
    int const *buffer_i = &i; // buffer points to a const int i
    cout << buffer_i <<" " << *buffer_i <<  endl;
    //  *buffer = 4; // NOT allowed
    buffer_i++; // allowed
    cout << buffer_i << " " << endl;

    int j = 2;
    int* const buffer_j = &j; // buffer is a const pointer to an int
    cout << buffer_j <<" " << *buffer_j <<  endl;
    *buffer_j = 4; // allowed
    cout << buffer_j <<" " << *buffer_j <<  endl;
    //  buffer_j++; // NOT allowed

    int const k = 5;
    int const *const buffer_k = &k; // buffer is a const pointer to a const int
    cout << buffer_k <<" " << *buffer_k <<  endl;
    //  *buffer_k = 4; // NOT allowed
    //  buffer_k++; // NOT allowed
    return 0;
}
```

const in Function Arguments

- *const* keyword when applied to function argument means that the called function cannot modify the value of the argument, e.g.
  - void print_name(const int& number);

- *const* return value, e.g.

```
const char *strchr(const char *p, char c); /* strchr cannot modify the C-style string
    passed to it via char *p and returns a pointer to char that cannot be modified */
```

const in method signature

```
class TestClass {
    string hidden_attribute;
    public:
    void check_const_func() const {
        int i = 0;
        i++;
        cout << this->hidden_attribute << " " << i << endl;
        // hidden_attribute = "new"; // ERROR! cannot modify members in const function
    }
};
```

Range-*for* statement and *auto*

- In addition to C-style for and while statements, C++ provides a range-for statement

```
int a[] = {1, 5, 78, 34, 56};
int sum = 0;
for (auto x : a) { // for each x in a
    sum += x;
}
cout << sum << endl;
```

- auto declaration can be used wherever the compiler can infer type

```
string s1 = “abc”;
auto j = s1.find(“bc”, 0); // s1.find() returns an unsigned int
```

Function Overloading

```
class Animal {
    string name = "Unicorn";
    public:
    string sayName() const {
        return name;
    }
};

class Cat {
    string name = "Tom";
public:
    string sayName() const {
        return "I am " + name + " Cat";
    }
};

class Queen {
    string name = "Toodles";
public:
    string sayName() const {
        return "Hello Whatever!! I don't care .. who you are...";
    }
};
```

```
void sayHello(Cat& cat, Queen& queen) {
    cout << " Hello my Queen! " << cat.sayName() << "\n";
    cout << queen.sayName();
}

void sayHello(Queen& queen, Cat& cat) {
    cout << queen.sayName();
    cout << " Hello my Queen!" << cat.sayName() << "\n";
}

void sayHello(Animal& animal, Cat& cat) {
    cout << animal.sayName();
    cout << cat.sayName();
}

int main() {
    Cat cat;
    Queen queen;
    Animal animal;
    sayHello([&]cat, [&]queen);
    sayHello([&]queen, [&]cat);
    sayHello([&]animal, [&]cat);
}
```

Function Overloading Rules

- For overloaded functions, ***arguments must differ*** in
  - Number or
  - Order or
  - Type
- Different return types not enough for overloading

```
void set (float fnum, int inum) {…}
int set (float fnum, int inum) {…} // NOT allowed
```

Default Values for Arguments

```
#include <iostream>
using namespace std ;
void printit ( int x = 10, float y = 4.65) {
        cout << x << y ;
}
int main( ) {
        printit (15, 3.14); // 15, 3.14
        printit (25); // 25, 4.65
        printit( ); // 10, 4.65
}
```

- Default values can only occur at the end of parameter-list

  void bar(int 1 = 10, float k) {…}

  // NOT allowed
- Calling function must supply all non-default arguments

Classes and Objects

this Pointer

- Disambiguates references to members of the class from methods of the same class
- Available to all the methods in a class via implicit declaration
- Always points to the current object
- Usage of *this* pointer is optional
- *this* is a constant pointer
- Can't modify it during execution of the method
- It dies once control returns from the method





```
class FlawlessCat {
    string name;
    int age;
    char gender;
public:
    void setData(string name, int age, char gender){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    void sayHello(FlawlessCat visitor){
        cout << "Hello " << visitor.name;
    }
};
int main(){
    FlawlessCat tom;
    tom.setData("Thomas", 10, 'M');
    FlawlessCat toodles;
    toodles.setData("Toodles Galore", 2, 'F');
    tom.sayHello(toodles);
}
```







this Pointer

- Disambiguates references to members of the class from methods of the same class
- Available to all the methods in a class via implicit declaration
- Always points to the current object
- Usage of *this* pointer is optional
- *this* is a constant pointer
- Can't modify it during execution of the method
- It dies once control returns from the method

```
#include<iostream>
using namespace std;
class FlawlessCat {
    string name;
    int age;
    char gender;
public:
    void setData(string name, int age, char gender){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    void sayHello(FlawlessCat visitor){
        cout << "Hello " << visitor.name;
        cout << ", I am " << this->name << endl;
    }
};
int main(){
    FlawlessCat tom;
    tom.setData("Thomas", 10, 'M');
    FlawlessCat toodles;
    toodles.setData("Toodles Galore", 2, 'F');
    tom.sayHello(toodles);
}
```

Access Specifiers

- Access Specifiers
  - private, public
- Defaults
  - structure – all members are public
  - class – all members are private
- Access Mechanism
  - Public members are accessible from outside the class
  - Private members are NOT accessible from outside the class
  - Within a class any member can access any other member
- https://github.com/ahanapradhan/OOPs/blob/main/jerry_likes_cheese.cpp
  - What does this program do?
  - If we switch places between isYummy() and isYuck() in Cheese class, what happens?
  - If you want to know jerry's age, what to do?

Incomplete Types

- An **incomplete type** is a type whose **size or layout is unknown** to the compiler at a certain point.
- The compiler **cannot allocate memory** for objects of an incomplete type.
- Why do you need it? -- forward declarations in header files, circular dependencies.. Etc.

```
class MyClass;  // Forward declaration → incomplete type
struct Node;    // Incomplete type
```

```
class A { B* b; }; // Allowed (pointer to incomplete type)
class B { A a; };  // ❌ Error (object of incomplete type)
```

Incomplete Types (contd.)

- **Allowed:**
  - Pointers to incomplete type: `Cat* ptr;` ✅
  - References to incomplete type: `Cat& ref;` ✅
- **Not allowed:**
  - Objects of incomplete type as **members**: `Cat obj;` ❌
  - Arrays of incomplete type: `Cat arr[10];` ❌
  - Using size-dependent operations (e.g., `sizeof`) ❌

Incomplete Types (contd.)

- **Why allowed:** Compiler treats the class as **complete inside its own scope**, so it knows how to copy objects.

```cpp
#include <iostream>
using namespace std;

class Cat {
    public:
    //Cat jasper; Not allowed due to unknown memory layout
    void greet(Cat& other); // allowed
    void greet(Cat* other); // allowed
    void greet_joyfully(Cat other); // allowed
};

int main() {
    Cat cat1;
    Cat cat2;
    cat1.greet([&]cat2); // linking error due to greet method definition
    cat2.greet_joyfully(cat1); // linking error due to greet_joyfully method definition
}
```

[OOPs/incomplete_type.cpp at main · ahanapradhan/OOPs]

Interface

Interface vs Implementation

```
// interface file Cat.h
#ifndef CAT_H
#define CAT_H
#include <string>
using namespace std;

class Cat {
    string name;
    int speed;
public:
    ~Cat() {};
    string tellYourName() const;
    void sayHello() const;
    void changeNameByAffidavit(string name);
    void doRun();
};
#endif //CAT_H
```

```
/* File Cat.cpp – implementation */
 #include "Cat.h"
 #include <iostream>

 Cat::Cat(string name, int speed) {
        this->name = name;
        this->speed = speed;
 }
 string Cat::tellYourName() const {
        return this->name;
 }
 void Cat::sayHello() const {
        cout << this->name << " says Hello to You! " << endl;;
 }
 void Cat::changeNameByAffidavit(string name) {
        this->name = name;
 }
 void Cat::doRun() const {
        cout << this->name << " is running "<< this->speed << "km/hr!\n";
 }
```

```
// main.cpp
#include "Cat.h"
int main() {
    Cat cat("Tom", 10);
    cat.tellYourName();
    cat.changeNameByAffidavit("Jasper");
    cat.tellYourName();
}
```

Interface vs Implementation

- Interface of a class defines the organization of objects of that class
  - Similar to a type definition (typedef)
- Class interface is specified in a class header (.h/.hpp) file
- Member functions that are declared in the interface are defined in the implementation
- The implementation is specified in the source (.cpp) file
- Separation of implementation and interface is highly recommended for:
  - Better readability
  - Easier maintenance
  - Separate compilation of implementation and user code
- Users of a class only use its interface (class header file)

Interface vs Implementation (contd.)

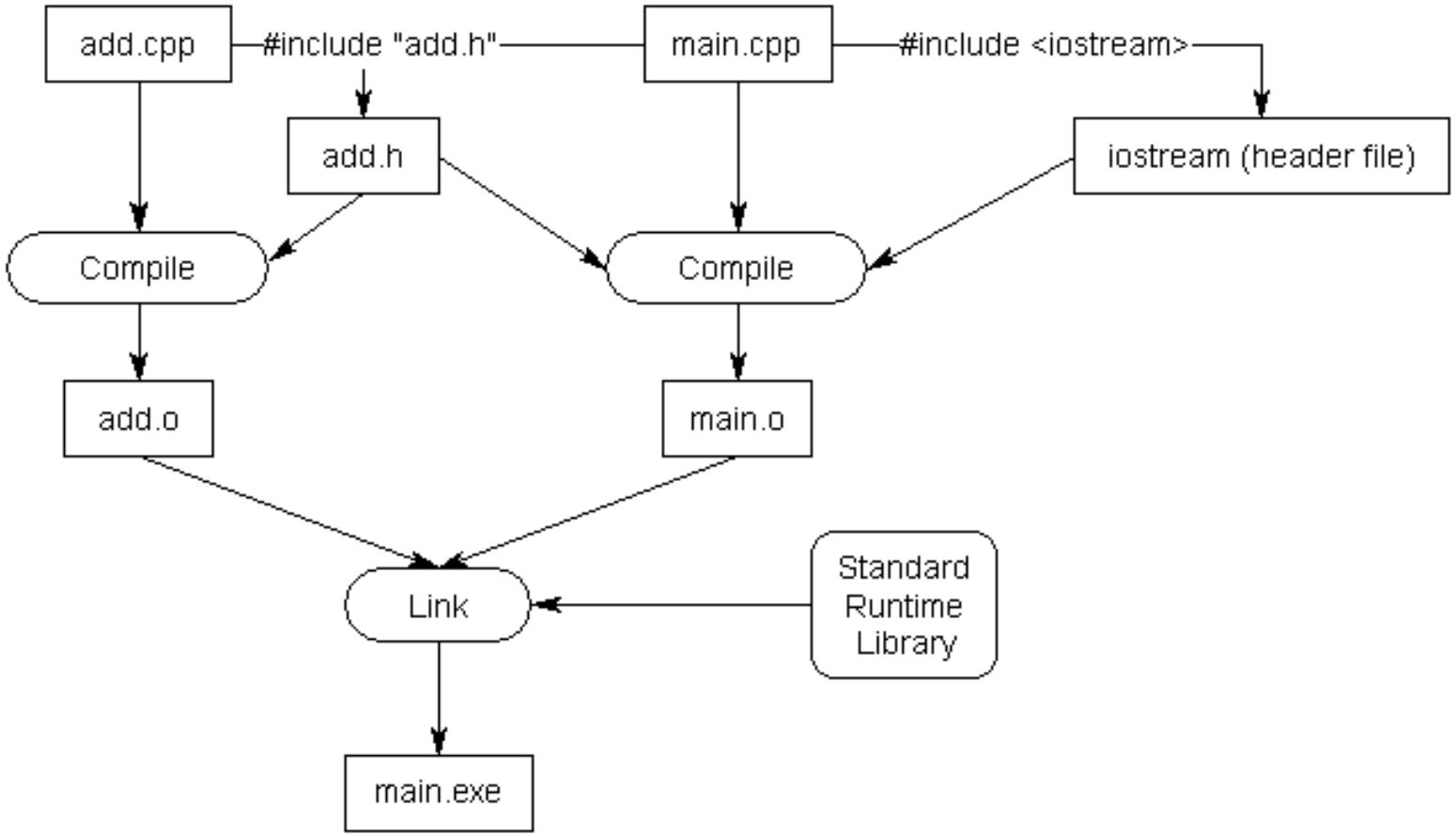


Image Courtesy: https://greenmon.dev/2018/11/24/cpp-header-file.html

Interface vs Implementation (contd.)

- Each of our examples has included iostream interface from iostream.h with

  ```
  # include <iostream>
  ```

- This allows the use of standard input and output objects (cin & cout) and operators (<< and >>) without knowing the implementation of input / output streams
- Source files containing the implementation of these objects and operators are pre-compiled and included in the C++ standard library
- Implementation source file also includes the same header file(s) to provide a consistent interface
- g++ compiler knows how to link standard library with the code we compile so we get to use the iostream facility without knowing how it is implemented
- Any change in the interface (header file) forces recompilation of implementation as well as all the source files that use that interface

Constructor and Destructor

Object Initialization

- At the time of declaration, e.g.



```
FlawlessCat tom {"Tom", 10, 'M'}; // if data members are public and types match
```

- Using initialization methods, e.g.

```
FlawlessCat tom;
```



```
tom.setData("Tom", 10, 'M'); // works with private data also
```

- Problem: what if users forgets to initialize?
- Establishing and maintaining “invariants” for a class of objects
  - Default values, e.g.



```
class FlawlessCat
{
    name = "Tom", age = 10, gender = 'M';
            …
}
```

- *What if more complex initialization is needed?*

Constructors

- Constructors are ***functions with same name as class names***
  - May use parameters needed to assign values to data members
  - Initializes objects of the class immediately on creation
- Constructors can be defined as regular class methods
- Constructors do not have return type



```
class FlawlessCat {
    string name;
    int age;
    char gender;
public:
    FlawlessCat(string name, int age, char gender){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
     …
};
int main(){
    FlawlessCat tom("Thomas", 10, 'M');
    FlawlessCat toodles("Toodles Galore", 2, 'F');
    tom.sayHello(toodles);
}
```









Constructors

- Constructors can be overloaded
  - Same function name can have different signatures
  - Signature must differ in the number or order or type of arguments
  - Return value is not part of the function signature
- Default constructor is provided by the compiler if no constructors are defined
- All object creation attempts must match one of the constructors, if defined

```
…

class FlawlessCat {
    string name;
    int age;
    char gender;
public:
    FlawlessCat(string name, int age, char gender){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    FlawlessCat(){
        this->name = "Tom";
        this->age = 10;
        this->gender = 'M';
    }
    …

};
int main(){
    FlawlessCat tom;
    FlawlessCat toodles("Toodles Galore", 2, 'F');
    tom.sayHello(toodles);
}
```

Conflicting Constructors

```
class FlawlessCat {
   …
public:
    FlawlessCat(string name, int age, char gender){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
    FlawlessCat(){
        this->name = "Tom";
        this->age = 10;
        this->gender = 'M';
    }
   FlawlessCat(string name="Tom", int age=5, char gender='M'){
        this->name = name;
        this->age = age;
        this->gender = gender;
    }
   …
};
int main(){
    FlawlessCat tom;
    FlawlessCat toodles("Toodles Galore", 2, 'F');
    tom.sayHello(toodles);
}
```

what is the problem here?

Destructor

- A destructor is a member function that is invoked implicitly when
  - the object goes out of scope (function or program) or
  - is explicitly destroyed by a call to delete
- A destructor has the same name as the class, preceded by a tilde (~)
- Default destructor is provided by the compiler if none is defined
- If there are pointers in class members pointing to other objects, then one needs to explicitly delete them to free memory
- **Good practice: Write destructors in case of use of pointers in classes**
  - **C++ does not have its own garbage collector**

```
class FlawlessCat {
    …
    vector<Trap*> traps;
public:
  FlawlessCat(string name, int age, char gender){
     …
     Trap* bananaPeel = new Trap();
     traps.push_back(bananaPeel);
  }
  ~FlawlessCat(){ // destructor
     for (auto& trap: traps){
        delete trap;
     }
     cout << "I do not have traps anymore!"
  }
  int getMyTrapCount(){
      return this->traps.size();
  }
};
int main(){
  FlawlessCat tom;
  tom.getMyTrapCount();
}
```

Namespaces

- Container for related classes and objects
- namespace std
  - Defined in iostream.h and related header files
  - Contains library classes – istream, ostream
  - Contains objects – cin, cout
- In the absence of “using namespace std”
  - std::cout << “hello world”;
  - std::cin >> greetings;

Defining and using a Namespace

- OOPs/namespaces_cat.cpp at main · ahanapradhan/OOPs